

# ALGORITMOS E PENSAMENTO COMPUTACIONAL



Estruturas de Dados Básicas (homogêneas)

- Introdução a estrutura de dados
- Arrays unidimensionais e multidimensionais

**Professor Marco Antonio**

**2º semestre 2026**

# Desafio de hoje!!!!!



Uma empresa deseja registrar os salários de cinco funcionários e, depois, exibi-los na mesma ordem em que foram informados.

## Problema

Como armazenar os cinco valores para utilizá-los novamente depois da leitura?

Uma solução poderia começar assim:

```
float salario1, salario2, salario3, salario4, salario5;
```

## Reflexão:

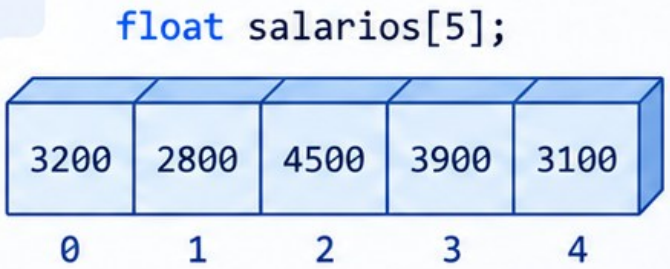
- E se fossem 50 funcionários?
- E se fossem 500?
- Como percorrer todos os salários usando uma estrutura de repetição?



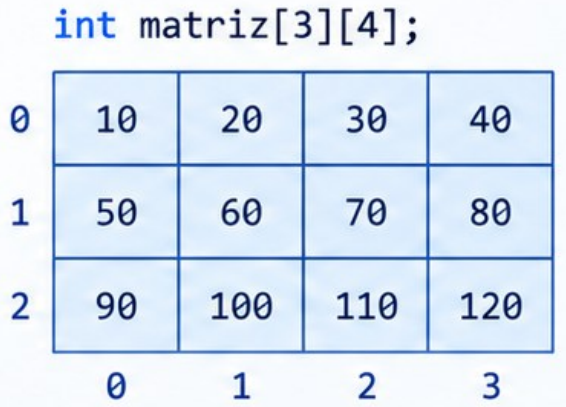
O problema não é apenas guardar os dados. Precisamos **organizar os dados para processá-los.**

# Conteúdos abordados nesta aula

Estruturas de Dados Básicas em C











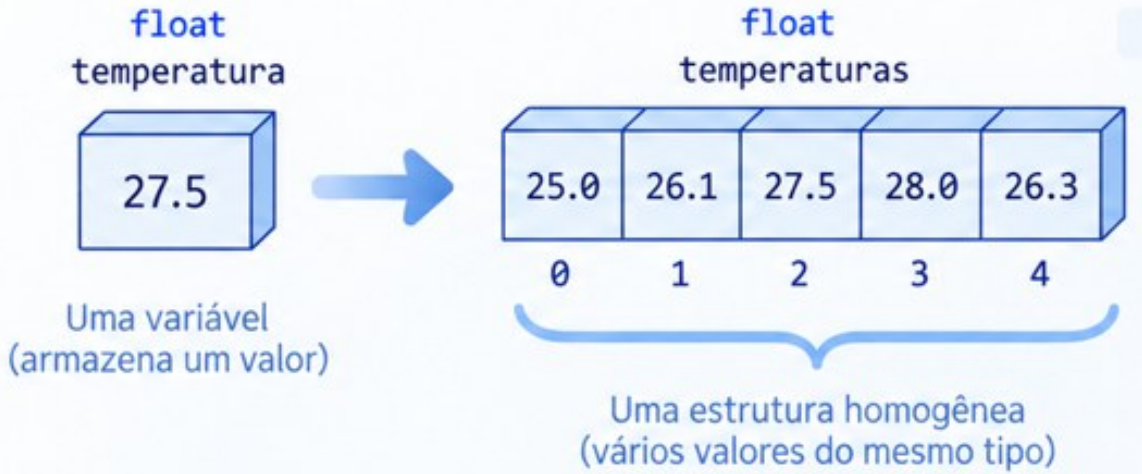


# Estruturas de dados

Uma variável comum armazena um valor de cada vez.

```
float temperatura = 27.5;
```

Quando vários valores do mesmo tipo representam elementos de um mesmo conjunto, podemos organizá-los em uma estrutura de dados homogênea.




## Estruturas homogêneas

Todos os elementos possuem o mesmo tipo.

**Exemplos:**

- notas de uma turma → float
- idades de alunos → int
- temperaturas diárias → float
- quantidade vendida por dia → int

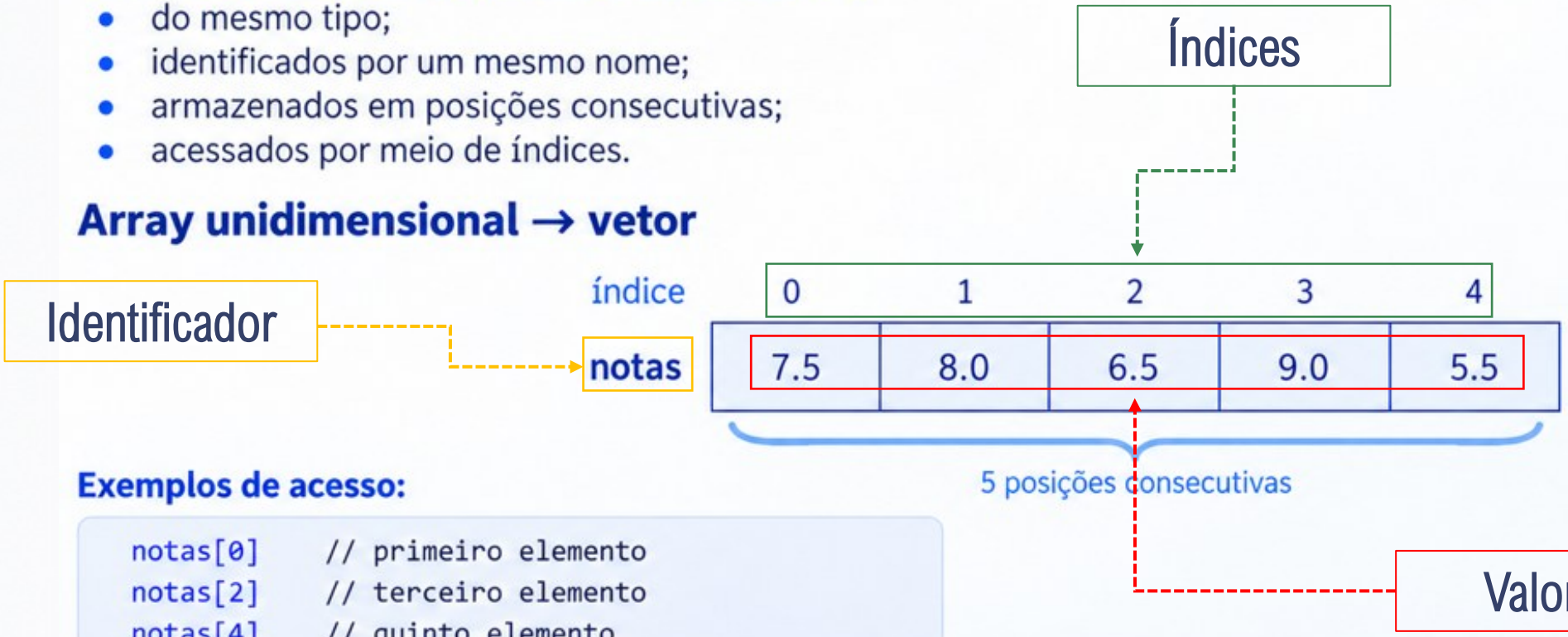
Na linguagem C, o **arrays** são estruturas fundamentais para organizar esses conjuntos de dados.

# O que é um array?

**Um array é um conjunto de elementos:**

- do mesmo tipo;
- identificados por um mesmo nome;
- armazenados em posições consecutivas;
- acessados por meio de índices.

**Array unidimensional → vetor**



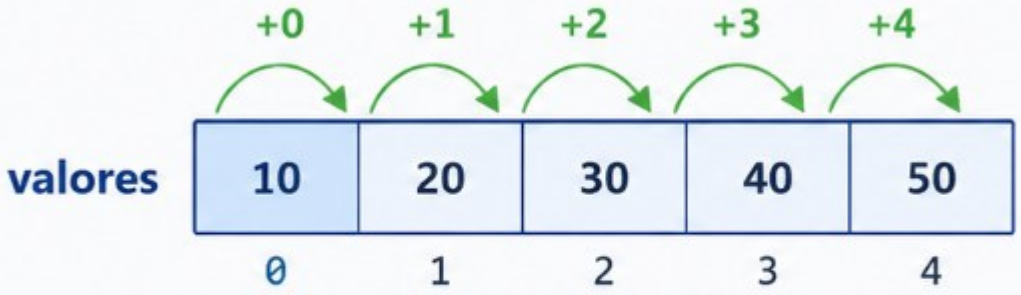

**Exemplos de acesso:**

```
notas[0]    // primeiro elemento
notas[2]    // terceiro elemento
notas[4]    // quinto elemento
```

Ideia-chave: **posição e conteúdo** são coisas diferentes.
O **índice** indica **onde** o valor está; o **conteúdo** é o valor (**o que**) armazenado.

# Por que começar por zero????

Podemos pensar no índice como um **deslocamento a partir do início** o array.




```
valores[0] → deslocamento 0
valores[1] → deslocamento 1
valores[2] → deslocamento 2
```

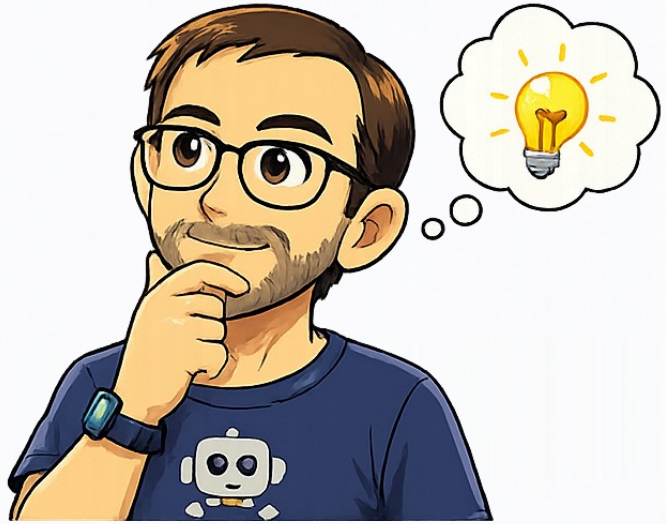
**Erro clássico**

```
int v[5];
v[5] = 20;   // ERRO: posição fora dos limites do vetor
```

Os índices válidos são **0, 1, 2, 3 e 4.**

**Atenção!!!**

Em C, não existe verificação automática de limites. Um acesso inválido produz **comportamento indefinido.**



# Tipos de arrays

Existem diversos tipos de arrays. Na linguagem C, os mais utilizados em contextos introdutórios são os arrays unidimensionais e os arrays multidimensionais. Eles organizam dados do mesmo tipo e permitem diferentes formas de representação.

## Array unidimensional → vetor

Organiza os elementos em uma única sequência, acessada por um único índice.

| índice | 0 | 1 | 2 | 3 | 4 |
|---|---|---|---|---|---|
| **valores** | 12 | 18 | 10 | 25 | 30 |

Exemplo em C:

```
int vendas[5] = {12, 18, 10, 25, 30};
```

## Array multidimensional → matriz

Organiza os elementos em linhas e colunas, acessados por mais de um índice.

| | coluna | 0 | 1 | 2 |
|---|---|---|---|---|
| linha | 0 | 10 | 20 | 30 |
| | 1 | 40 | 50 | 60 |
| | 2 | 70 | 80 | 90 |

Exemplo em C:

```
int notas[3][3] = {{10, 20, 30}, {40, 50, 60}, {70, 80, 90}};
```

Também é possível trabalhar com arrays de mais dimensões, mas os casos mais comuns no início são 1D e 2D.

Nesta aula, o foco será em **vetores e matrizes**, que são estruturas homogêneas mais importantes para começar a programar em C.

# Declarando e inicializando vetores em C

## Declaração

Reserva espaço na memória para um array do tipo especificado.

```
int idades[5];
float notas[4];
```

## Inicialização completa

Declara o array e já atribui um valor para cada elemento.

```
int valores[5] = {10, 20, 30, 40, 50};
```

## Tamanho inferido pelo compilador

O compilador determina automaticamente o tamanho do array com base na quantidade de elementos informados.

```
int valores[] = {10, 20, 30, 40, 50};
```

## Inicialização com zeros

Inicializa todos os elementos do array com zero.

```
int valores[5] = {0};
```



Cuidado!!!

```
int valores[5];
```

Se um **array** não for inicializado, seus elementos possuem valores indeterminados.

# Acessando e alterando elementos

## Acessando e modificando elementos

Podemos acessar um elemento do array utilizando seu índice e atribuir um novo valor, ou usar o valor de outros elementos.

```
int pontos[4] = {10, 20, 30, 40};

pontos[0] = 15;
pontos[2] = pontos[0] + pontos[1];
```

## Resultado:

Após as modificações, o array fica com os seguintes valores:

| | 0 | 1 | 2 | 3 |
|---|---|---|---|---|
| **pontos** | 15 | 20 | 35 | 40 |

## Leitura de uma posição

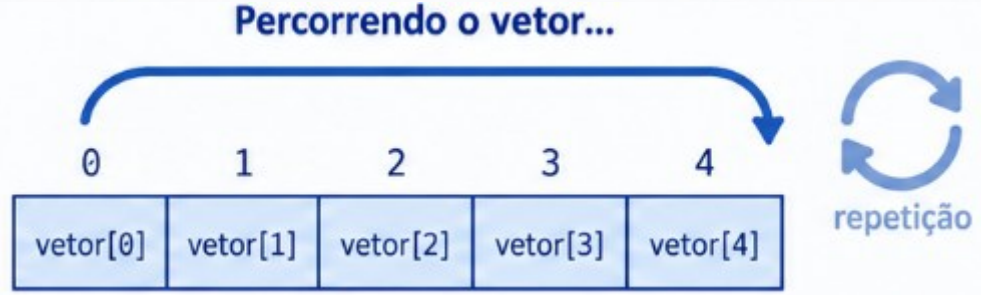
Podemos ler um valor para uma posição específica do array utilizando o scanf.

```
scanf("%d", &pontos[3]);
```

Observe o **&**: o **scanf** precisa do endereço da posição que receberá o valor.

# Percorrendo um vetor

Um **vetor** torna-se realmente útil quando o combinamos com uma **estrutura de repetição.**




### Pseudocódigo

```
para i de 0 até TAM - 1
    leia(vetor[i])
fim-para
```

### Linguagem C

```
for (i = 0; i < TAM; i++) {
    scanf("%d", &vetor[i]);
}
```

### Para exibir os valores

```
for (i = 0; i < TAM; i++) {
    printf("%d ", vetor[i]);
}
```

**Padrão importante:**
Um vetor unidimensional normalmente é percorrido com um laço de repetição (`for`)

# Hora de praticar!!






# Vamos praticar????



O setor administrativo de uma pequena empresa precisa registrar o salário de quatro funcionários para gerar uma conferência dos dados cadastrados.

## Problema

1 Desenvolva um programa em C que:

- leia o salário de 4 funcionários;
- armazene os valores em um vetor;
- exiba os salários na mesma ordem em que foram digitados.

## Caso de teste

**Entrada:**

```
Digite o salário do 1º funcionário: 1500
Digite o salário do 2º funcionário: 3500
Digite o salário do 3º funcionário: 2600
Digite o salário do 4º funcionário: 3100
```

**Saída:**

```
Funcionário 1: R$ 1500.00
Funcionário 2: R$ 3500.00
Funcionário 3: R$ 2600.00
Funcionário 4: R$ 3100.00
```

# Por que armazenar antes de processar?

Depois que os salários estão armazenados, podemos realizar novos processamentos **sem pedir os dados novamente.**

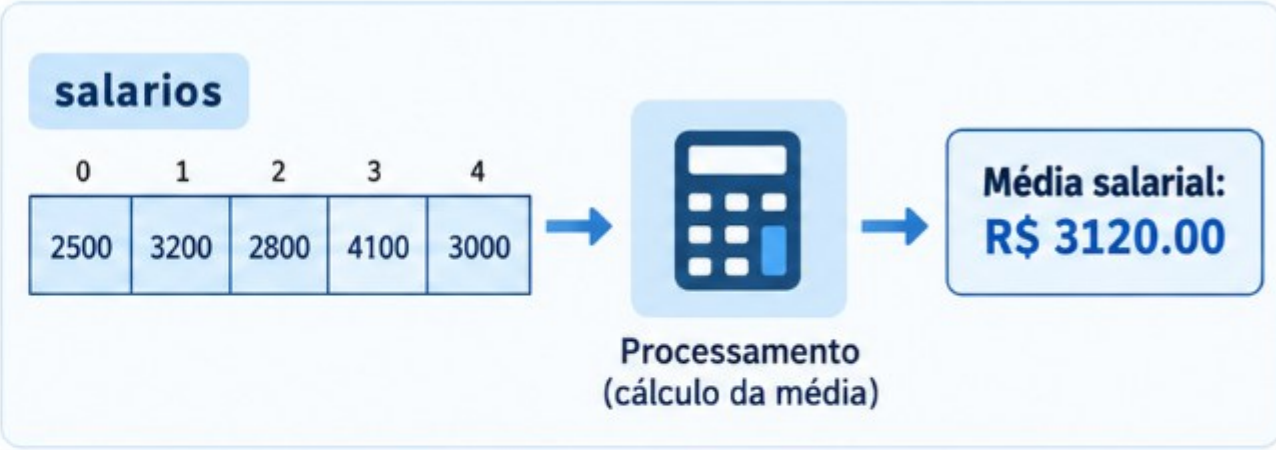
**Exemplo:** calcular a média salarial

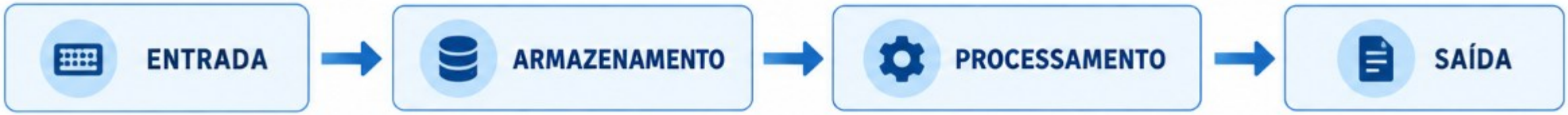
```
float soma = 0.0f;
float media;

for (i = 0; i < TAM; i++) {
    soma += salarios[i];
}

media = soma / TAM;

printf("Média salarial: R$ %.2f\n", media);
```




Um mesmo conjunto de dados pode ser processado várias vezes.

# Vamos praticar????



Um sistema de monitoramento registra oito medições de desempenho de um equipamento ao longo do dia. O analista deseja identificar o comportamento geral dessas medições.

## Problema

2 Desenvolva um programa em C que:

- leia 8 valores reais e armazene-os em um vetor;
- calcule a média dos valores;
- determine quantos valores ficaram **acima da média**.

## Caso de teste

```
Entrada:
Digite o xº valor:
Ex.: 6 8 5 9 7 10 4 7

Saída:
Média: 7.00
Valores acima da média: 3
```

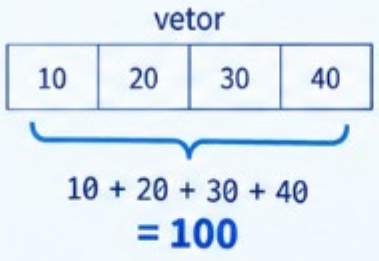
# Padrões de processamento em vetores

Muitos problemas com vetores **são combinações de poucos padrões fundamentais.**

## Acumulação

Soma os elementos do vetor (ou calcula outra acumulação).

```
soma += vetor[i];
```




## Contagem

Conta quantos elementos satisfazem uma condição.

```
if (vetor[i] > media)
    contador++;
```




## Maior valor

Encontra o maior elemento do vetor.

```
maior = vetor[0];
for (i = 1; i < TAM; i++) {
    if (vetor[i] > maior)
        maior = vetor[i];
}
```




## Menor valor

Encontra o menor elemento do vetor.

```
menor = vetor[0];
```




**Boa prática:** para maior/menor, use um elemento válido do próprio vetor como valor inicial.

# Vamos praticar????

Uma empresa deseja obter indicadores rápidos sobre os salários de um pequeno setor para apoiar uma análise interna.

## Problema

3 Desenvolva um programa em C que leia e armazene os salários de 6 funcionários e, depois:

1. calcule a média salarial;
2. determine o maior salário;
3. conte quantos salários estão abaixo da média.

### Caso de teste

```
Entrada:
Digite o salário do x° funcionário:
Ex.: 1800 2500 2200 3100 1500 2700

Saída:
Média salarial: R$ 2300.00
Maior salário: R$ 3100.00
Salários abaixo da média: 3
```

# Quando uma dimensão não é suficiente

Imagine que precisamos armazenar as notas de:

- 3 estudantes;
- em 4 avaliações.

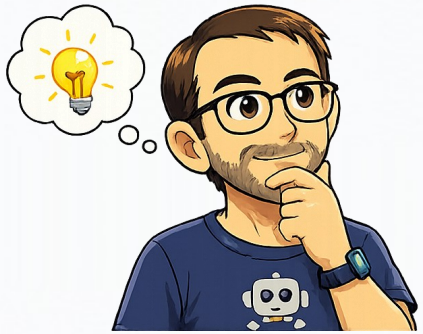
**Com vetores separados:**

```
float aluno1[4];
float aluno2[4];
float aluno3[4];
```

Organizando em linhas e colunas

**Mas os dados possuem naturalmente uma organização em linhas e colunas.**

| | Avaliação | | | |
|---|---|---|---|---|
| | 1 | 2 | 3 | 4 |
| **Aluno 1** | 8.0 | 7.0 | 9.0 | 6.0 |
| **Aluno 2** | 6.0 | 5.0 | 7.0 | 8.0 |
| **Aluno 3** | 9.0 | 8.0 | 10.0 | 9.0 |



**Pergunta:** qual estrutura representa melhor essa organização?

# Matrizes: arrays com duas dimensões

Uma **matriz** é um array bidimensional organizado em:

- linhas;
- colunas.

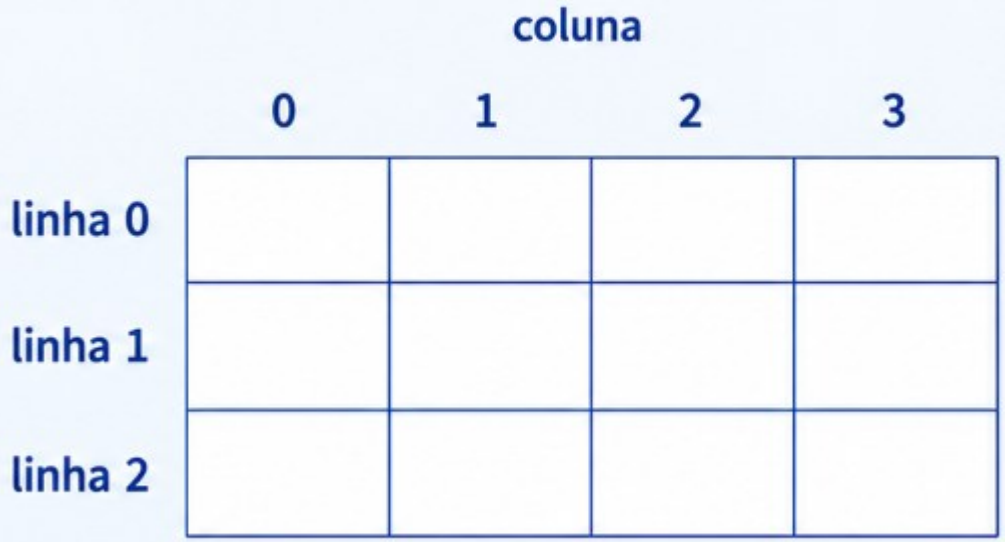
**Declaração:**

```
float notas[3][4];
```

**Acesso a um elemento:**

```
notas[linha][coluna]
```

**Representação:**




Em uma matriz, cada elemento é identificado por **dois índices**: um para linha outro para coluna.

# Declarando e inicializando matrizes

## Declaração

```
int vendas[3][4];
```

São 3 × 4 = 12 elementos do tipo **int**.

## Inicialização

```
int vendas[3][4] = {
    {10, 12,  9, 11},
    { 8,  7, 10,  9},
    { 5,  6,  4,  7}
};
```

## Acesso

```
vendas[0][0]   // 10
vendas[1][2]   // 10
vendas[2][3]   // 7
```

## Representação na memória

| linha \ coluna | 0 | 1 | 2 | 3 |
|---|---|---|---|---|
| 0 | 10 | 12 | 9 | 11 |
| 1 | 8 | 7 | 10 | 9 |
| 2 | 5 | 6 | 4 | 7 |

Em cada dimensão, os índices começam em zero (linhas e colunas).

Em cada dimensão, os **índices** também começam por **zero**.

# Como percorrer uma matriz?

Para percorrer duas dimensões, usamos **duas estruturas de repetição aninhadas.**

**Pseudocódigo**

```
para linha de 0 até LINHAS - 1
    para coluna de 0 até COLUNAS - 1
        leia(matriz[linha][coluna])
    fim-para
fim-para
```

**Linguagem C**

```c
for (i = 0; i < LINHAS; i++) {
    for (j = 0; j < COLUNAS; j++) {
        scanf("%d", &matriz[i][j]);
    }
}
```




Padrão:
laço externo (i): percorre as linhas;
laço interno (j): percorre as colunas.

# Vamos praticar????



Uma cafeteria universitária acompanha a quantidade vendida de três produtos durante quatro dias. Cada linha representa um produto e cada coluna representa um dia.

## Problema

4 Desenvolva um programa em C que:

- leia as vendas de 3 produtos durante 4 dias;
- calcule o total vendido de cada produto;
- calcule o total geral de itens vendidos.

## Caso de teste

**Entrada:**

```
Digite a qte do x° produto no y° dia:
Ex.: 10 12 9 11
     8 7 10 9
     5 6 4 7
```

**Saída:**

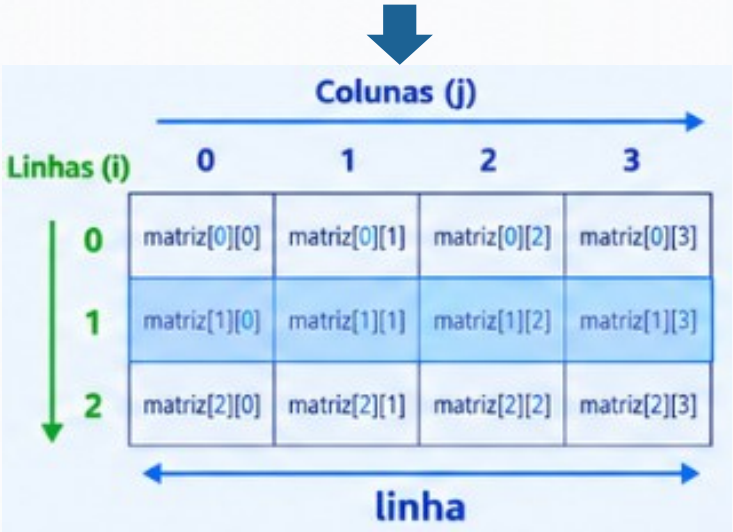
```
Produto 1: 42 unidades
Produto 2: 34 unidades
Produto 3: 22 unidades
Total geral: 98 unidades
```

# Processando linhas e colunas

Uma matriz pode ser analisada de maneiras diferentes dependendo do problema.

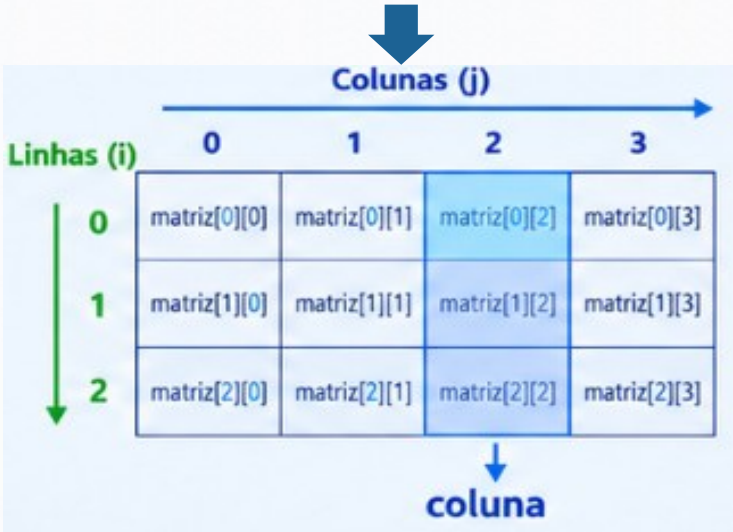
## Somar uma linha

```
soma = 0;
for (j = 0; j < COLUNAS; j++) {
    soma += matriz[linha][j];
}
```



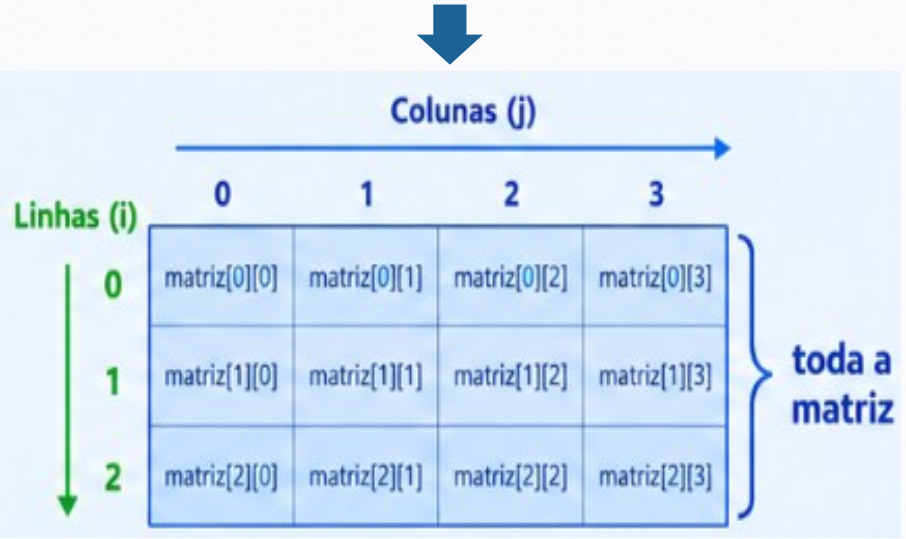

## Somar uma coluna

```
soma = 0;
for (i = 0; i < LINHAS; i++) {
    soma += matriz[i][coluna];
}
```




## Processar toda a matriz

```
for (i = 0; i < LINHAS; i++) {
    for (j = 0; j < COLUNAS; j++) {
        // processa matriz[i][j]
    }
}
```



# Reflexão e discussão




**Pergunta:**

no exemplo da cafeteria, o que representa a soma de uma coluna?

# Desafio!!!!



Crie um vetor para os dias da semana e outro para os produtos e personalize o exercício anterior.

## Exemplo:

```
const char *produtos[] = {"café", "coxinha", "bolo", };
const char *dias_semana[] = {"seg", "ter", "qua"};
```



### Atenção:

- Uma **variável** comum guarda um valor. Um **ponteiro** (*) guarda um endereço.
- Em C, **& obtém o endereço** de uma variável e *** acessa o valor** armazenado nesse endereço.

### Caso de teste

```
Entrada:
Digite a qte de cafés vendidos na seg: 10
Digite a qte de cafés vendidos na ter: 5
...
Digite a qte de bolos vendidos na qua: 6

Saída:
Cafés: 42 unidades
Bolos: 34 unidades
Coxinhas: 22 unidades
Total geral: 98 unidades
```

# Vamos praticar????



Uma disciplina registra quatro avaliações para três estudantes. A coordenação deseja calcular a média individual e identificar o estudante com maior média.

## Problema

5 Desenvolva um programa em C que:

- leia as 4 notas de 3 estudantes;
- calcule e exiba a média de cada estudante;
- identifique o estudante com a maior média.

## Caso de teste

**Entrada:**

```
Digite a xª nota do y° estudante:
Ex.: 8 7 9 6
     6 5 7 8
     9 8 10 9
```

**Saída:**

```
Média do estudante 1: 7.50
Média do estudante 2: 6.50
Média do estudante 3: 9.00
Maior média: estudante 3 - 9.00
```

# Decompondo a solução

## Etapa 1: armazenar as notas

```c
for (i = 0; i < ALUNOS; i++) {
    for (j = 0; j < AVALIACOES; j++) {
        scanf("%f", &notas[i][j]);
    }
}
```

## Etapa 2: calcular a média de cada linha

```c
for (i = 0; i < ALUNOS; i++) {
    soma = 0.0f;

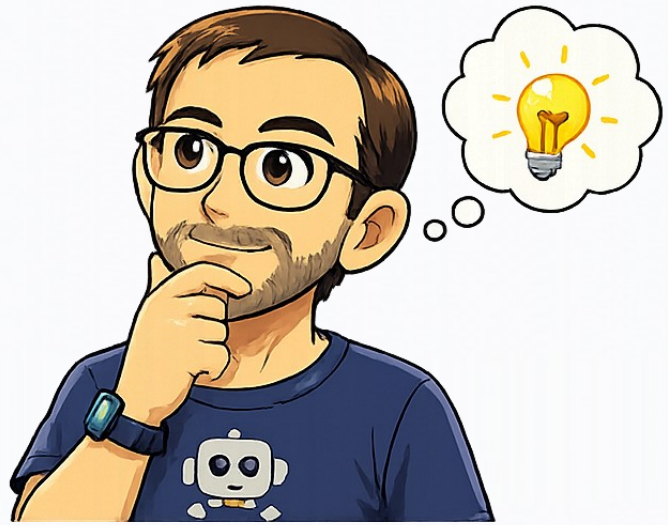
    for (j = 0; j < AVALIACOES; j++) {
        soma += notas[i][j];
    }

    media = soma / AVALIACOES;

    if (i == 0 || media > maiorMedia) {
        maiorMedia = media;
        melhorAluno = i;
    }
}
```

### Pensamento computacional

- **Decomposição:** leitura → cálculo das médias → busca da maior média.
- **Reconhecimento de padrão:** cada aluno corresponde a uma linha processada da mesma forma.



# Arrays com mais de duas dimensões

A linguagem C permite arrays com três ou mais dimensões.

**Exemplo:**

```c
int estoque[2][3][4];
```

Uma interpretação possível:

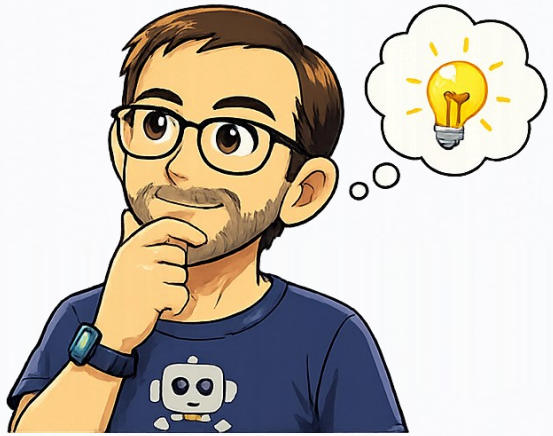
- 2 lojas;
- 3 setores por loja;
- 4 produtos por setor.

**Acesso:**

```c
estoque[loja][setor][produto];
```

**Importante!!!**

- A quantidade de índices acompanha a quantidade de dimensões.
- Nesta etapa, o foco principal permanece nos arrays de **uma e duas dimensões.**



# Vamos praticar????



Uma estação meteorológica registra a temperatura máxima de cada dia de uma semana.

## Problema

6 Leia 7 temperaturas e armazene-as em um vetor. Calcule:

- a média semanal;
- a maior temperatura;
- a menor temperatura;
- quantos dias ficaram acima da média.

## Caso de teste

```
Entrada:
Digite a temperatura da [seg]:
Ex.: 25 27 26 30 28 24 29

Saída:
Média: 27.00 °C
Maior temperatura: 30.00 °C
Menor temperatura: 24.00 °C
Dias acima da média: 3
```

# Atenção: arrays em C não protegem você

### 1. Não existe .length

```
vetor.length    // não existe em C
```

Use uma constante ou mantenha o tamanho em uma variável.

### 2. Não existe new para declarar arrays comuns

```
int vetor[10];
```

### 3. Arrays locais não recebem zero automaticamente

```
int vetor[10] = {0};  // inicializa explicitamente
```

### 4. Índice inválido não gera necessariamente uma mensagem de erro

```
vetor[10] = 5;  // comportamento indefinido se o
                tamanho for 10
```

### 5. Não podemos copiar arrays com atribuição simples

```
// destino = origem;  // não copia arrays em C
```

A cópia deve ser feita elemento a elemento.

# Síntese: o que aprendemos?

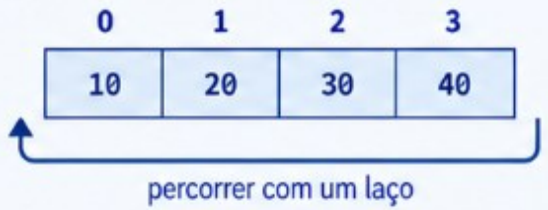
## 1 Vetores

```
tipo nome[TAMANHO];
```

**Um índice:**

```
vetor[i]
```

Normalmente percorrido com um laço.




## 2 Matrizes

```
tipo nome[LINHAS][COLUNAS];
```

**Dois índices:**

```
matriz[i][j]
```

Normalmente percorrida com laços aninhados.

| i ↓ \ j → | 0 | 1 | 2 | 3 |
|---|---|---|---|---|
| 0 | 1 | 2 | 3 | 4 |
| 1 | 5 | 6 | 7 | 8 |
| 2 | 9 | 10 | 11 | 12 |

## 3 Padrões que continuam aparecendo

- acumular;
- contar;
- comparar;
- buscar maior/menor;
- filtrar;
- percorrer dados sistematicamente.

## 4 Reflexão final

O array não resolve o problema sozinho. Ele **organiza os dados** para que o algoritmo possa processá-los de maneira sistemática.

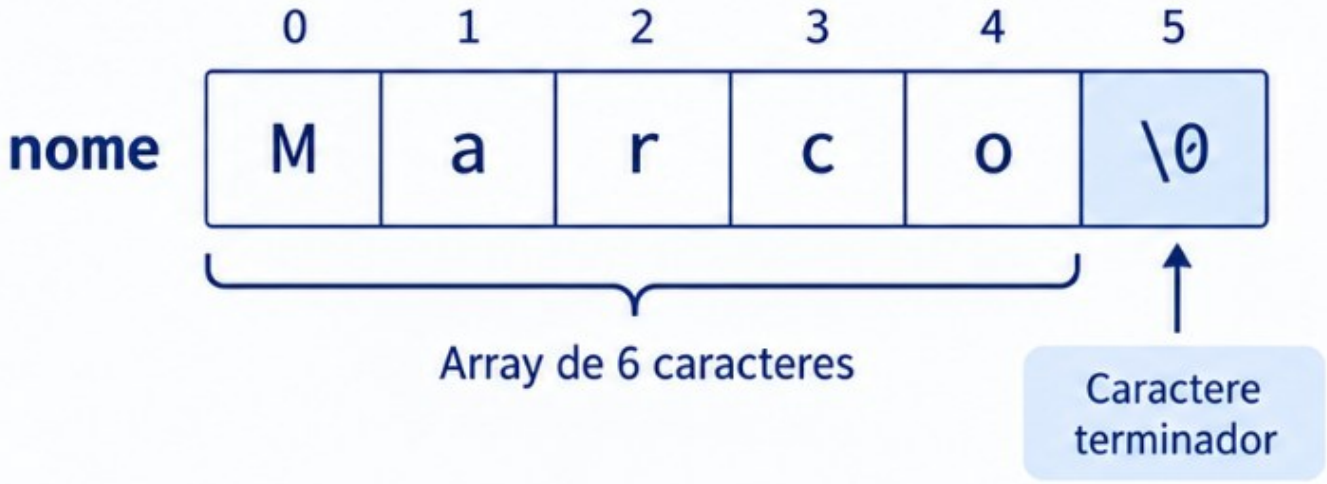
# String também é vetor em C

Uma **string** é um array de caracteres.

```
char nome[6] = "Marco";
```

**Explicação:**

- nome[0] = 'M'
- nome[1] = 'a'
- nome[2] = 'r'
- nome[3] = 'c'
- nome[4] = 'o'
- nome[5] = '\0'




Em C, uma string é um vetor de `char` terminado por `'\0'`.

# Uso simples de strings

## Exemplo com `scanf`:

```c
char nome[20];

printf("Digite seu nome: ");
scanf("%19s", nome);

printf("Nome digitado: %s\n", nome);
```

## Alternativa usando `fgets`:

```c
char nome[20];

printf("Digite seu nome: ");
fgets(nome, 20, stdin);

printf("Nome digitado: %s\n", nome);
```

## Observações curtas

- string é um vetor de caracteres;
- `%s` é usado para exibir strings;
- ao ler com `scanf("%s", ...)`, **não usamos &nome;**
- `scanf("%s", nome)` lê apenas até o primeiro espaço;
- `fgets()` é mais adequada quando queremos ler textos com espaços.

## Comparação na prática

Entrada do usuário: `Ana Paula Silva`

`scanf("%s", nome)` → Resultado em nome: `A n a` ...
lê apenas até o primeiro espaço

`fgets(nome, 20, stdin)` → `A n a P a u l a S i l v a \n`
lê a linha completa (com espaços)

# Principais funções de String

## 1 Tamanho da String:

**strlen():** Conta quantos caracteres a **string** possui (**não** conta o **\0**).

```c
char texto[] = "C";
int tamanho = strlen(texto);  // Retorna 1
```

Sintaxe: `strlen(string)`




## 2 Comparar Strings (case sensitive):

**strcmp():** Compara duas **strings** caractere por caractere (considerando maiúsculas e minúsculas diferentes).

```c
if (strcmp("abc", "abc") == 0) {
    /* Iguais! */
}
if (strcmp("abc", "ABC") != 0) {
    /* Diferentes! */
}
```

- Sintaxe: `strcmp(string1, string2)`
- **Retorna** 0 quando as strings são idênticas.

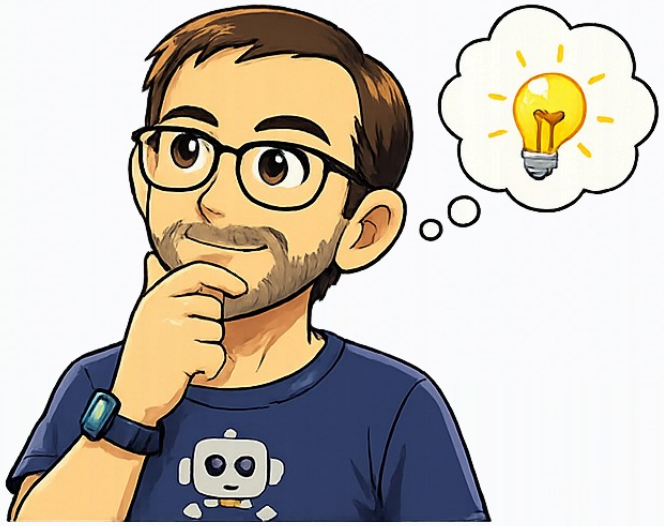
# Principais funções de String

## 3 Comparar Strings (ignore case):

**`strcasecmp()`:** Funciona igual ao `strcmp()`, mas ignora se a letra é maiúscula ou minúscula (ótimo para sistemas de busca).

```
if (straasepp("Qua", "qua") == 0) {
    /* Retorna 0 (Iguais) */
}
```

- **Sintaxe:** `strcasecmp(string1, string2)`
- **Retorna** 0 quando as strings são iguais, ignorando maiúscula/minúscula.



# Alguma dúvida????



Universidade Cruzeiro do Sul

## CRÉDITOS

Material elaborado por:

Prof. Marco Antonio Sanches

Material base elaborado por:

- Prof. Alcides
- Prof. Lédon
- Prof. Ana Paula
- Profa. Cristiane

Última atualização: 2025/2

## CONTATOS




marco.sanches@cruzeirodosul.edu.br



marco-antonio-sanches-anastácio-601b9123